1. Aim

To diagnose a performance limitation in the monthly statement program from Q4, redesign the program using a more suitable string-building method, and validate the improvement using the same input data and measurements.

2. Selected Program and Reason for Selection

The selected program is Q4 – Building Long Text: + or join(). It generates a monthly statement by combining one text line for every call or SMS. The original version uses repeated += inside a loop. This is a suitable program for redesign because the amount of text grows as the number of statement records increases.

3. A. Diagnose the Bottleneck

The main work is performed inside the loop that appends each new item to the statement. The growing string is repeatedly combined with the next part. Since Python strings are immutable, concatenation may create a new string and copy previously accumulated characters. As the statement grows, more characters may need to be copied.

The count that grows is the number of statement lines and the total number of output characters. The original code may repeat copying of earlier text for each new line. The actual bottleneck should be confirmed by timing the original program at different input sizes.

4. B. Define the Design Objective

The objective is to generate exactly the same statement text for 1,000, 10,000 and 200,000 records while reducing the average statement-building time, especially for the largest input. The redesigned program must preserve every item and its separator.

Success will be measured by comparing average execution time over three runs for both methods, and by checking that their output strings are identical.

5. C. Design the Revised Solution

The redesign collects each formatted item in a list and joins all parts once. This uses the property that lists can store multiple string references and str.join() can combine them efficiently. Additional memory is required for the list of parts and the final output string. The trade-off is higher temporary memory use compared with writing directly to a file.

Pseudocode:

```
Create an empty list called parts
For each statement item:
    format the item as a string
    append the formatted item and separator to parts
Join all elements of parts into one string
Return the completed statement
```

6. Original Program – Repeated +=

```
def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text
```

This is the original method from Q4. It builds the statement by repeatedly appending the next item to the current string.

7. Redesigned Program – List and join()

```
def build_join(parts):
    pieces = []
    for p in parts:
        pieces.append(p + ";")
    return "".join(pieces)
```

The redesigned method stores each completed line in a list. The join() call then combines all parts into the final statement.

8. D. Predict Before Implementing

I predict that the list-and-join method will be faster for large statements because it combines the parts in one join operation instead of repeatedly rebuilding the accumulated string. I also predict that both methods will produce exactly the same output length and content. The redesigned method will use extra temporary memory to store the list of parts.

9. E. Implement and Validate Using the Same Inputs

The following program tests both methods with identical data. It checks output equality and measures each method three times. Run it on your computer and use the printed averages as your experimental evidence.

```
import time

def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text

def build_join(parts):
    pieces = []
    for p in parts:
        pieces.append(p + ";")
    return "".join(pieces)

def average_time(function, data, repeats=3):
    times = []
    result = None
    for _ in range(repeats):
        start = time.perf_counter()
        result = function(data)
        elapsed = time.perf_counter() - start
        times.append(elapsed)
    return result, sum(times) / len(times)
```

```
for n in (1_000, 10_000, 200_000):
    items = [f"Item{i}:Rs{i % 50}" for i in range(n)]

    original, plus_avg = average_time(build_plus, items)
    redesigned, join_avg = average_time(build_join, items)

    print("Lines:", n)
    print("Same output:", original == redesigned)
    print("Output length:", len(original))
    print(f"Original += average: {plus_avg:.6f} seconds")
    print(f"Redesigned join average: {join_avg:.6f} seconds")
    if join_avg > 0:
        print(f"Speed ratio (+=/join): {plus_avg / join_avg:.2f}x")
    print()
```

10. Experimental Results Table

Enter the actual averages printed by the program. Do not use invented or copied timings because execution time depends on the computer and Python environment.

Input lines	Original += average (s)	Redesigned join average (s)	Outputs identical?
1,000	Fill in measured value	Fill in measured value	Yes / No
10,000	Fill in measured value	Fill in measured value	Yes / No
200,000	Fill in measured value	Fill in measured value	Yes / No

To calculate the improvement, divide the original average time by the redesigned average time. A result greater than 1 means the redesigned method is faster. For example, a ratio of 2.0 means the redesigned method took half the time of the original.

11. F. Defend the Final Design

The redesigned method is successful if it produces identical statement text and reduces the measured average time, particularly for 200,000 lines. The list-and-join method has linear time complexity in the total output size, while repeated concatenation can require much more character copying in the general case.

The new cost is extra memory for the list of string parts and the final joined string. If the statement is extremely large and does not need to be held in memory as one string, writing each line directly to a file may be preferable. The original += approach may still be acceptable for very small statements where simplicity matters and performance is not important.

12. Time and Space Complexity

Method	Time complexity	Space complexity	Reason
Repeated += (general case)	Up to $O(N^2)$ character copying	O(N) final output	Accumulated text may be copied repeatedly.
List + join()	O(N)	O(N)	Parts are stored and combined in one join operation.

N represents the total number of characters in the statement. Actual CPython performance may vary because some += operations can be optimised, so the measured results should be discussed alongside the theoretical complexity.

13. Execution Evidence to Attach

- Screenshot showing the original and redesigned programs running.
- Output confirming that both generated statements are identical.
- Measured averages for 1,000, 10,000 and 200,000 lines.
- The raw three timing measurements for each method and input size.
- A graph comparing the average execution times of the two methods.

14. Conclusion

The repeated += method in the monthly statement program was selected because its performance can become limited as the number of records grows. The redesigned list-and-join method avoids repeatedly rebuilding the complete string and is expected to perform better for large inputs. The final decision must be supported by measured execution times and an output-equality check. The redesigned method uses additional temporary memory, while direct file writing may be more suitable when the statement is too large to keep in memory.